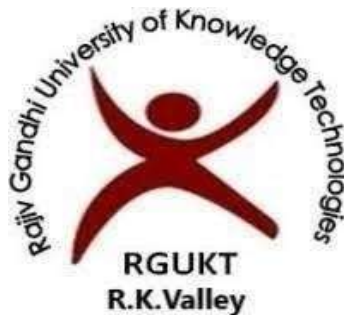


Minor Project Report
On
Emotion-Based Music Recommendation System

Submitted By
M.Sathish Kumar Reddy
R210747

Under the Guidance of
Mr. Satyanandaram N
MSIT (IIIT Hyderabad)
Lecturer
Computer Science and Engineering

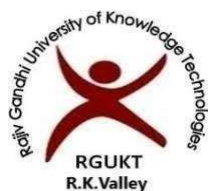


Rajiv Gandhi University of Knowledge and Technologies

RGUKT RK Valley, Kadapa, 516330

Department of Computer Science and Engineering

2025-2026



*Rajiv Gandhi University of Knowledge Technologies
RK Valley, Kadapa (Dist.), Andhra Pradesh - 516330*

CERTIFICATE

This is to certify that the project report entitled “Emotion-Based Music Recommendation System” being submitted by M.Sathish Kumar Reddy (R210747) under my guidance and supervision and is submitted to DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING in partial fulfilment of requirements for the award of Bachelor of Technology in Computer Science and Engineering during the academic year 2025-2026 and it has been found worthy of acceptance according to the requirements of the University.

Project GUIDE

Mr. Satyanandaram N
MSIT (IIIT Hyderabad)
Lecturer
RGUKT, RK VALLEY

HEAD OF THE DEPARTMENT

Dr. Ch. Ratna Kumari
Mtech (UOH), Ph.D.(JNTU H)
Assistant Professor
RGUKT, RK VALLEY

EXTERNAL EXAMINER

DECLARATION

I hereby declare that this project work entitled “**Emotion-Based Music Recommendation System**”, submitted to the **Department of Computer Science and Engineering**, is a genuine work carried out by me under the guidance of **Mr. Satyanandaram N.** This project is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)**.

I further declare that this work has not been submitted elsewhere for the award of any other degree or diploma other than as specified above.

M.Sathish Kumar Reddy
R210747

ACKNOWLEDGEMENT

I would like to express my sense of gratitude and respect to all those people behind the screen who guided, inspired, and helped me crown all my efforts with success.

I express my gratitude to my project guide

Mr. Satyanandaram N MSIT for his valuable guidance at all stages of the study, advice, constructive suggestions, supportive attitude, and continuous encouragement, without which this project would not have been possible.

I extend my heartfelt thanks to **Dr. Ch. Ratna Kumari M.Tech, Ph.D., Head of the Department, Computer Science and Engineering**, for her continuous support and for providing us with the resources and academic environment necessary for this project.

I sincerely acknowledge the academic support of **Mr. D. Ramesh Kailas, M.Tech** Dean of Academics, RGUKT RK Valley.

I am also grateful to **Prof. A. V. S. S. Kumara Swami Gupta M.Tech, Ph.D. Director of RGUKT RK Valley**, for his encouragement and for fostering a culture of research and innovation.

My sincere thanks go to the **faculty members of the Department of Computer Science and Engineering** for their guidance, and academic support.

Finally, I express my heartfelt gratitude to my parents for being a constant source of encouragement and inspiration, helping me keep my morals high.

With sincere Regards
M.Sathish Kumar Reddy
R210747

Table of Contents

S.No	Content	Page No
1	Abstract	6
2	Introduction	7 – 8
2.1	Problem Statement	7
2.2	Objective	8
3	Technologies Used	8 - 9
4	App Architecture	11 – 14
4.1	Core Modules	10 - 16
4.2	Feature Modules	17
4.3	Platform Specific Entry Points	18
5	Data preprocessing	19
6	Model architecture	20 – 22
7	Proposed system workflow	22 – 23
8	Experimental setup and Hyperparameters	23 - 25
9	Challenges faced and solutions	25 – 27
10	Future Scope	27 - 28
11	ML Pipeline Execution Stages	29 -31
12	Web Application Preview	32-39
13	Conclusion	40

Abstract:

In recent years, advancements in Artificial Intelligence (AI) and Computer Vision have significantly improved the ability of machines to understand and interpret human emotions. One of the most impactful applications of this technology is in enhancing user experience through personalized systems. This project presents an Emotion-Based Music Recommendation System that detects human emotions from facial expressions and suggests music accordingly, creating a more engaging and adaptive interaction between users and digital platforms.

To improve model performance and generalization, multiple training iterations are conducted, and techniques such as dropout and early stopping are applied to prevent overfitting. Once trained, the model is saved and integrated into a real-time application environment. The system accepts user input in the form of images, processes them, and predicts the corresponding emotional state along with confidence scores.

Based on the detected emotion, the system recommends suitable music tracks, leveraging external services such as music streaming APIs. This creates a personalized and emotionally adaptive music experience for users. The project demonstrates the practical implementation of deep learning in emotion recognition and recommendation systems, highlighting key skills such as dataset preparation, model training, real-time prediction, and system integration. Overall, this system contributes to the development of intelligent human-computer interaction and showcases how AI can be effectively utilized to enhance digital entertainment platforms by delivering emotion-aware content.

Furthermore, the proposed system can be extended to various real-world applications such as mental health monitoring, smart assistants, and adaptive user interfaces. By continuously improving model accuracy and incorporating advanced technologies, this approach has the potential to revolutionize personalized digital experiences and make human-computer interactions more intuitive, responsive, and emotionally aware.

Introduction :

Problem Statement

With the rapid advancement of Artificial Intelligence and Computer Vision, systems are increasingly capable of interpreting human emotions through facial expressions. However, most existing music platforms provide generalized recommendations without considering the user's current emotional state. This often leads to a lack of personalization and reduced user engagement, as the content delivered does not align with the user's mood or feelings.

Traditional recommendation systems rely mainly on user history, preferences, or trending data, which fail to capture real-time emotional context. As a result, these systems are unable to dynamically adapt to the user's present state of mind. There is a growing need for an intelligent system that can analyze facial expressions in real time and accurately identify emotions to provide context-aware recommendations.

To address this challenge, this project proposes a deep learning-based Emotion-Based Music Recommendation system that captures user images and predicts emotions using a trained Convolutional Neural Network (CNN). The system processes facial features, classifies the detected emotion, and suggests appropriate music tracks accordingly. By integrating emotion recognition with music recommendation, the project aims to deliver a personalized, efficient, and user-friendly solution that enhances user experience and bridges the gap between human emotions and digital entertainment.

Objective:

- To develop a deep learning model capable of accurately recognizing human emotions from facial expressions.
- To design and implement a Convolutional Neural Network (CNN) architecture for multi-class emotion classification.
- To prepare and preprocess a facial expression dataset by converting images to grayscale and resizing them to a uniform resolution (48×48).
- To train the model using efficient optimization techniques, including the Adam optimizer and multiple epochs, to improve accuracy and generalization.
- To design a user-friendly system that captures user images and provides real-time emotion-based music recommendations.
- To enhance user experience by providing personalized music recommendations based on detected emotions.

- To evaluate the model's performance using metrics such as accuracy, precision, recall, and confusion matrix for real-world applicability.

Technologies Used:

The development of the **Emotion-Based Music Recommendation System** involved a combination of deep learning frameworks, backend and frontend technologies, dataset handling tools, and model training utilities. The following technologies were used to build and deploy the complete solution:

Deep Learning & Model Development

- **TensorFlow / Keras:** Core deep learning framework used to build, train, and fine-tune the Convolutional Neural Network (CNN) for emotion recognition.
- **OpenCV:** Used for image preprocessing, face detection, and feature extraction.
- **PIL (Python Imaging Library):** Used for image loading, resizing, and preprocessing.
- **Scikit-learn:** Used for evaluation metrics such as confusion matrix and accuracy measurement.
- **Matplotlib:** Used for visualization of training performance including loss and accuracy graphs.
- **NumPy:** Used for numerical computations and efficient array operations during model training and prediction.

Dataset Handling & Cloud Integration

- **Google Colab:** Cloud-based GPU environment for training the deep learning model efficiently.
- **Local Dataset Storage:** Used to store and manage facial expression datasets organized into emotion-based folders.
- **Custom Data Processing Pipeline:** Implemented to load images dynamically, apply preprocessing techniques such as resizing and normalization, and prepare data for model training.
- **Model Checkpoints & Storage:** Used to save trained models and intermediate results for future use and evaluation.

Web Development

- **Backend:** Django framework used to develop the backend system for handling requests, managing server-side logic, and integrating the emotion prediction model. It processes user inputs and returns the predicted emotion along with results.
- **Frontend:** HTML, CSS, and JavaScript are used to build a simple and responsive user interface for capturing user images, sending data to the backend, and displaying the detected emotion and recommended music.

Model Training

- **Convolutional Neural Network (CNN):** Used for facial emotion recognition by extracting features from input images and classifying them into multiple emotion categories.
- **Adam Optimizer:** Adaptive learning rate optimization algorithm used for efficient and faster convergence during training.
- **Early Stopping:** Technique used to prevent overfitting by monitoring validation performance and stopping training at the optimal point.
- **Data Normalization & Resizing:** All images are resized to 48×48 pixels and normalized to ensure consistent input and improved model performance.

Version Control & Development Tools

- **Git & GitHub:** Used for version control, project management, and maintaining code history.
- **Jupyter Notebook:** Interactive environment used for model development, experimentation, and testing.

App Architecture:

The following section presents a comprehensive explanation of the core modules involved in the system, detailing their internal workings, responsibilities, and interactions.

Core Modules:

The Emotion-Based Music Recommendation System is architected around a collection of core modules that form the backbone of the application. These modules operate cohesively yet independently, providing a robust structure for tasks such as dataset handling, image preprocessing, model training, prediction, backend processing, and integration with the user interface. Each module is designed to maintain modularity, scalability, and clarity, following standard software engineering practices.

1. Dataset Handling Module:

The Dataset Handling Module is responsible for preparing the facial expression images for model training. Since the performance of the emotion recognition model depends heavily on the quality of input data, this module ensures that the dataset is properly structured, preprocessed, and suitable for efficient deep learning operations.

1.1. Dataset Composition

The dataset includes:

- A large collection of facial images categorized into seven emotions: angry, disgust, fear, happy, neutral, sad, and surprise.
- The dataset is divided into training and testing sets, with a combined total of more than 36,000 images used for model development and evaluation.

1.2. Class Imbalance & Its Challenges

The dataset contains images across multiple emotion classes, but the distribution may not always be perfectly balanced. This can introduce certain challenges:

- The model may become biased toward emotions with a higher number of samples.
- Less represented emotions may not be learned effectively.
- To address these challenges, preprocessing techniques such as data normalization, augmentation, and careful dataset splitting are applied. These methods help improve model performance, ensure better learning across all emotion categories, and enhance overall prediction accuracy.

1.3. Custom Dataset Class

A custom data handling approach is implemented to efficiently load and process images during training:

- Images are dynamically loaded from dataset folders at runtime.
- Corrupted or unreadable images are handled safely to avoid training interruptions.
- All images are converted to grayscale format for consistency with the model input.
- Emotion labels are automatically assigned based on folder structure.
- Supports easy scalability for adding more emotion categories or extending the dataset.

1.4. DataLoader Configuration:

Efficient data loading and batching are managed using suitable configurations:

- Batch size: 32
- Shuffling: Enabled during training to improve model generalization.
- Parallel loading: Utilized to speed up data processing.
- Memory optimization: Applied to improve performance during training.

2. Image Preprocessing Module:

Preprocessing is essential to ensure uniform quality and optimize inputs for the CNN-based emotion detection model.

2.1. Normalization:

The pixel intensity normalization is performed to scale values into a standard range, ensuring:

- Improved training stability
- Faster convergence
- Better model performance

2.2. Resize Strategy:

Images are resized to 48×48 pixels based on:

- Model input requirement
- Reduced computational complexity
- Maintaining essential facial features for accurate emotion detection
- Ensuring consistent input dimensions across all samples

2.3. Transformation Pipeline:

The transformation pipeline includes:

1. Resize
2. Convert to Tensor
3. Normalize
4. Random Shuffle (at batch-level)

This pipeline is executed dynamically during training.

2.4. Why No Augmentation

Although data augmentation is a common technique, in this system:

- Facial features must remain consistent for accurate emotion detection
- Augmentation may distort important emotional patterns
- Maintaining original data improves model reliability

Thus, augmentation is avoided to preserve feature integrity.

3. Model Architecture Module:

CNNs are highly effective for image-based tasks due to their ability to automatically extract spatial features. They provide efficient learning, high accuracy, and are suitable for real-time applications.

3.1. Why CNN Architecture

The Convolutional Neural Network (CNN) used in this project is well-suited for facial emotion recognition as it can effectively capture spatial features from images. CNNs automatically learn important patterns such as facial expressions, edges, and textures, which are crucial for distinguishing between different human emotions. The chosen architecture provides:

- Lightweight and efficient design suitable for real-time applications
- Fast training and inference
- High efficiency
- High accuracy in image classification tasks
- Strong capability in feature extraction from facial images
- Wide applicability in domains such as emotion detection, object recognition
- Medical imaging

3.2. Transfer Learning Advantages

The model is trained on a well-structured dataset of facial expressions, allowing it to learn meaningful patterns effectively. The training process offers:

- Faster convergence using optimized learning techniques
- Improved accuracy through multiple training epochs
- Robust feature extraction for identifying subtle emotional differences
- Better generalization to handle unseen real-time inputs

The model is trained efficiently, reducing computational cost while maintaining good performance.

3.3. Output Layer Configuration

The final layer of the CNN model is designed for multi-class emotion classification. The architecture is configured as follows:

- Dense layer with appropriate neurons corresponding to emotion classes
- Dropout layer used to reduce overfitting by randomly deactivating neurons during training
- Softmax activation function to generate probability scores for each emotion

The model outputs one of the seven emotion classes:

- Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise

Each prediction is accompanied by a confidence score, enabling reliable emotion-based music recommendations.

4. Training Module

This module focuses on the training process of the Convolutional Neural Network (CNN) used for facial emotion recognition. It involves multiple stages including data feeding, loss computation, optimization, and performance monitoring to ensure accurate emotion classification.

4.1. Epoch-Based Learning Process

The model is trained for multiple epochs, where each epoch represents one complete pass over the training dataset. This iterative process allows the model to gradually learn and improve its ability to recognize facial expressions. fundamental visual patterns (shapes,

edges, colors, gradients), the training primarily focuses on fine-tuning the deeper layers to learn domain-specific features.

Each epoch consists of:

1. **Forward Pass** – Input images are passed through the CNN to generate emotion predictions
2. **Loss Calculation** – The predicted output is compared with actual labels
3. **Backward Pass** – Gradients are computed using backpropagation
4. **Parameter Update** – Model weights are updated to minimize error

This process enables the model to learn meaningful facial features and improve prediction accuracy over time.

4.2. Loss Function & Optimization Strategy:

The model uses categorical cross-entropy loss, which is suitable for multi-class emotion classification problems. This function measures the difference between predicted probabilities and actual emotion labels.

The model uses the Adam optimizer, chosen for the following reasons:

- Adaptive learning rates per parameter
- Faster convergence compared to traditional optimizers
- Stable performance during training
- Optimal for transfer-learning on medium-sized datasets
- Efficient handling of complex image data

Adam's ability to adjust the learning rate dynamically for each parameter helps stabilize training, especially during the initial epochs when the model heavily modifies pretrained layers to adapt to the new classification task. This combination ensures efficient and stable model training.

4.3. Gradient Descent and Backpropagation

Backpropagation is the mathematical foundation that allows neural networks to learn. During each iteration, gradients are computed for every layer based on how much they contributed to the prediction error. These gradients flow backwards from the output layer toward the initial convolution layers.

Backpropagation is used to update the weights of the CNN by calculating gradients of the loss function with respect to each parameter.

In this model:

- Initial layers capture basic features such as edges and textures
- The deeper layers undergo more significant modifications
- Deeper layers learn complex facial patterns
- Final layers focus on emotion classification.

This layered learning approach improves overall model accuracy.

4.4. Handling Class Imbalance During Training

The dataset may contain unequal representation of different emotions, which can affect model performance. To handle this:

- Shuffling ensures randomization in every epoch
- Balanced DataLoaders maintain equal representation
- Dropout layers reduce overfitting to majority class features
- Balanced dataset splitting is used

These techniques improve model generalization and fairness across all emotion classes.

4.5. Early Stopping for Overfitting Prevention

Early stopping is implemented to monitor validation performance during training. If validation accuracy does not improve for a set number of epochs, training is stopped.

Early Stopping mechanism that monitors the validation loss at the end of each epoch.

If the validation loss fails to improve for 5 consecutive epochs, the training process halts automatically. This ensures:

- Reduced training time
- Prevention of overfitting
- Enhanced generalization to unseen data

Early stopping is particularly beneficial because pretrained networks can over-adapt quickly to small datasets.

4.6. Batch Processing and Parallelism

A batch size of 32 is used for training, which provides a balance between computational efficiency and stable learning.

- Faster training through parallel computation
- Efficient memory utilization
- Smooth gradient updates
- Stable learning dynamics

Batch processing enables parallel computation of gradients, significantly speeding up training. Furthermore, the DataLoader uses optimizations like:

- Multiple worker threads for fast image loading
- Pinned memory for fast CPU-to-GPU data transfer

This results in smooth, bottleneck-free training even on mid-range hardware.

4.7. Performance Monitoring and Metric Logging

Throughout the training process, key metrics are tracked for evaluation:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy
- Underfitting (high loss, low accuracy)
- Overfitting (training accuracy high, validation accuracy low)
- Learning stabilization (loss plateau)

These metrics help in analyzing model performance and identifying issues like underfitting or overfitting.

4.8. Convergence Behavior and Observations

During the experiments, the following observations were made:

- The model converged rapidly due to transfer learning
- Training accuracy increased consistently across epochs
- Validation accuracy stabilized by mid-training
- Loss curves showed smooth downward trends

Such convergence indicates that the dataset is high-quality and the feature differences between AI and real images are distinguishable enough for CNNs.

4.9. Additional Improvements

To further enhance performance, the system can incorporate:

- Hyperparameter tuning for better accuracy
- Larger and more diverse datasets
- Real-time optimization for faster predictions
- Integration with advanced recommendation systems

These improvements can make the system more robust and scalable for real-world deployment.

5. Evaluation Module:

The Evaluation Module is responsible for measuring how well the emotion recognition model performs after training. Once the training phase is completed, the model is tested on the validation dataset, which contains facial images that were not used during the learning process. This ensures that the model can accurately predict emotions for new and unseen inputs rather than memorizing the training data.

During evaluation, key metrics such as validation loss and validation accuracy are calculated. These metrics provide insight into how effectively the model performs on real-world data. A lower validation loss and higher accuracy indicate better model performance. To visualize this, graphs of loss and accuracy are plotted across epochs, showing how the model improves during training.

Additionally, a confusion matrix is generated to analyze the classification performance across all emotion classes. This helps identify which emotions are predicted correctly and where misclassifications occur. Sample predictions are also displayed to compare actual emotions with predicted outputs visually.

Overall, the Evaluation Module ensures that the trained model is reliable, accurate, and ready for deployment in real-time applications.

6. Feature Modules:

The Emotion-Based Music Recommendation System is organized into multiple feature-specific modules, each responsible for a specific functionality. These modules help maintain a clean structure and make the system easy to manage and extend.

6.1. Image Capture & Upload Module (Frontend):

This module allows users to capture or upload an image using the web interface. It includes UI components for accessing the camera or selecting files, basic validations, and preview functionality. Once the image is selected, it is sent to the backend using an API request.

6.2. Emotion Prediction Module (Backend):

This module receives the image through the Django backend, preprocesses it using the same steps applied during training, and passes it to the trained CNN model. The model predicts the emotion and returns the result along with a confidence score.

6.3. Results Display Module (Frontend):

After receiving the prediction from the backend, this module displays the detected emotion and confidence level clearly on the user interface. It may also trigger music recommendations based on the predicted emotion.

6.4. Visualization Module (ML Notebook):

This module generates visual representations such as accuracy graphs, loss curves, and confusion matrices. These visualizations help in analyzing model performance and are useful for documentation.

6.5. Model Export & Loading Module:

This module handles saving the trained model (e.g., .h5 format) and loading it into the backend system. It ensures that the model is ready for prediction whenever the application runs.

Overall, these feature modules work together to provide a smooth workflow—from capturing an image to detecting emotion and recommending music.

7. Platform-Specific Entry Points:

The system operates across two main platforms: the backend (Django server) and the frontend (web interface). Each platform has its own entry point to ensure smooth interaction between the user and the model.

7.1. Backend Entry Point (Django):

The backend entry point initializes the Django application, loads the trained emotion detection model, and configures the required routes and settings. It ensures that the system is ready to process incoming requests efficiently. It also initializes the Emotion prediction endpoint, which handles incoming image files. By loading the model at startup, the backend ensures fast and efficient inference without repeated model reloads during requests.

7.2. Frontend Entry Point (Web Application):

The frontend entry point initializes the user interface built using HTML, CSS, and JavaScript. It handles user interactions such as capturing images, sending requests to the backend, and displaying results. Emotion prediction is displayed to the user. The frontend entry point forms the user-facing layer of the system.

7.3. Integration Between Platforms

Both frontend and backend work together seamlessly:

- The frontend provides a user-friendly interface to capture user images
- The backend processes the images and predicts emotions
- The frontend displays the detected emotion and recommended music instantly

Data Preprocessing:

1. Dataset Overview:

The dataset used in this project consists of AI-generated and real-world images collected from multiple publicly available sources. The AI-generated images include outputs from generative models and online repositories, while real images are obtained from verified photography datasets and natural image collections. Since the data is collected from diverse environments, it includes variations in lighting, textures, objects, and scene composition. This diversity enables the model to learn generalized features rather than focusing on a limited set of visual patterns.

2. Image Resizing and Normalization:

Before feeding images into the model, it is necessary to standardize their dimensions and pixel values. All images are resized to a resolution of 225×225 pixels to maintain consistency while preserving important visual details. Normalization is applied to scale pixel intensities into a uniform range, which helps the neural network process inputs more effectively. This step reduces the impact of variations such as brightness, contrast, and noise, ensuring stable and efficient training for both AI-generated and real images.

3. Handling Class Imbalance:

Class imbalance is a common challenge in classification problems, especially when one class has significantly more samples than the other. In this project, AI-generated images are more abundant compared to real images, which can bias the model toward predicting the majority class. To address this issue, undersampling is applied to reduce the number of AI images, ensuring both classes have a balanced representation. Although this reduces the overall dataset size, it helps the model learn distinguishing features more effectively and improves classification fairness.

4. Train–Validation Split:

The dataset is divided into training and validation sets using a structured approach to ensure proper model evaluation. An 80–20% split is used, where the majority of data is utilized for training and the remaining portion is reserved for validation. This allows the model to learn effectively while also being tested on unseen data. Random sampling is applied during the split to avoid clustering of similar images, ensuring that both sets represent the overall dataset distribution.

5. Batch Processing with DataLoaders:

Efficient data loading and batching are implemented to optimize the training process. The PyTorch DataLoader is used to load images in batches and apply preprocessing operations in parallel, which significantly improves training speed. A batch size of 32 is used to balance computational efficiency and memory usage. Shuffling is enabled during training to ensure that each epoch presents data in a different order, preventing the model from learning sequence-based patterns. Additionally, consistent batch processing helps stabilize training and improves convergence.

Model Architecture:

1. Introduction to CNN-Based Classification:

Convolutional Neural Networks (CNNs) are specifically designed to learn and extract hierarchical visual features from images, making them highly effective for image classification tasks. Unlike traditional machine learning approaches that rely on manual feature extraction, CNNs automatically identify important patterns such as edges, textures, shapes, and complex structures through convolutional layers, pooling operations, and non-linear activations. In this project, CNNs play a crucial role in distinguishing between AI-generated and real images by capturing subtle differences in texture, noise distribution, and structural details that are often difficult for the human eye to recognize. This ability makes CNNs a powerful choice for solving complex visual classification problems like AI vs Real image detection.

2. Choice of CNN Model:

A custom Convolutional Neural Network (CNN) is used in this project to perform image classification. The architecture is designed to balance accuracy and computational efficiency, making it suitable for real-time applications. The model consists of multiple convolutional layers, pooling layers, and fully connected layers that work together to extract meaningful

features from images. This CNN is capable of capturing subtle differences between AI-generated and real images, such as texture patterns and noise distribution.

3. Training Approach:

The model is trained using a supervised learning approach, where labeled images (AI and Real) are provided as input. During training, the network learns to map image features to their corresponding class labels. The learning process focuses on identifying distinguishing patterns between synthetic and real images, allowing the model to make accurate predictions on unseen data.

4. Modified Classification Layer:

The final layer of the CNN model is specifically designed for binary classification. A dropout layer is included before the output layer to reduce overfitting by randomly deactivating neurons during training. The final dense (linear) layer produces two output values corresponding to the two classes: AI-generated and Real images. This simplified structure ensures efficient and accurate classification.

5. Optimization Setup:

The Adam optimizer is used due to its adaptive learning rate capabilities, which help in faster and more stable convergence during training. It adjusts the learning rate for each parameter individually, making it effective for handling complex and noisy datasets. CrossEntropy loss is chosen as the loss function because it measures the difference between predicted probabilities and actual class labels effectively in classification tasks. Together, this combination ensures smooth training, efficient gradient updates, and improved overall model performance.

Model Testing & Validation Techniques:

1. Validation Strategy:

Validation ensures that the model generalizes beyond the training data. By evaluating the model on a separate validation dataset after each epoch, the system avoids overfitting and ensures that the learned patterns are applicable to unseen images. This strategy allows continuous monitoring of model performance and supports decisions such as adjusting learning rates or applying early stopping when required.

2. Performance Metrics:

Accuracy is used as a primary metric to measure how many predictions are correctly classified. However, relying only on accuracy may not provide a complete picture. Therefore, validation loss is also monitored, as it reflects the confidence and consistency of the model's predictions. A model with high accuracy but high loss may still produce uncertain outputs, making it important to evaluate both metrics together.

3. Loss and Accuracy Curves:

Plotting training and validation loss and accuracy curves helps in visualizing the learning behavior of the model over epochs. If validation loss increases while training loss decreases, it indicates overfitting. In this project, the curves showed a stable trend, demonstrating that the model learned effectively without significant overfitting. These visualizations also help in selecting the best-performing model.

4. Confusion Matrix Analysis:

The confusion matrix is used to evaluate the performance of the AI vs Real image classification model by showing the number of correct and incorrect predictions for each class. In this binary classification task, it helps determine how many AI-generated images are correctly identified and how many real images are misclassified (and vice versa). This analysis is important because misclassifying real images as AI or AI images as real can impact the reliability of the system. By studying the confusion matrix, the model's weaknesses can be identified and improvements can be made.

5. Sample Prediction Comparison:

Sample prediction visualization is used to examine how the model performs on actual images. A set of test images is passed through the model, and the predicted labels (AI or Real) are compared with the true labels. This helps in understanding whether the model is correctly identifying visual patterns such as textures, noise, and image structures. It also helps in identifying difficult cases where the model may fail, such as highly realistic AI images or low-quality real images, providing insights for further enhancement.

Proposed System Workflow / Methodology:

1. Workflow Overview:

The AI vs Real Image Detection System follows a structured workflow that connects data acquisition, preprocessing, model training, and real-time deployment into a unified pipeline. The workflow ensures that images pass through consistent processing steps to maintain

accuracy and reliability. Each stage of the system contributes to improving the model's ability to recognize subtle differences between AI-generated and real images.

2. Training Workflow:

The training workflow begins with collecting images from AI-generated sources (such as generative models and online datasets) and real images from verified repositories. These images undergo preprocessing steps including resizing to 225×225 pixels, normalization, and balancing using undersampling techniques. After preprocessing, the dataset is split into training and validation sets using an 80–20 ratio. The data is then loaded in batches using PyTorch's DataLoader for efficient processing. The custom CNN model is trained over multiple epochs, where it learns important visual features and adjusts its weights to minimize classification error and improve prediction accuracy.

3. Inference Workflow:

During inference, the user uploads an image through the frontend interface built using HTML, CSS, and JavaScript. The image is validated, converted into the required format, and sent to the Django backend. The backend applies the same preprocessing steps used during training to maintain consistency. The processed image is then passed to the trained CNN model, which generates prediction probabilities for "AI" or "Real." The backend formats the result into a structured response and sends it back to the frontend, where it is displayed to the user instantly.

4. System Data Flow:

Data flows smoothly between modules:

- User → Frontend Upload
- Frontend → Django Backend Endpoint
- Backend → Preprocessing → CNN Model → Prediction
- Prediction → Frontend UI

This structured flow ensures accuracy, reliability, and consistent behavior in both training and real-time usage.

5. End-to-End Pipeline Summary:

Overall, the system integrates dataset preparation, model training, frontend interaction, backend processing, and result visualization into a unified pipeline. This workflow ensures that the trained CNN model performs consistently from development to deployment, allowing users to easily verify whether an image is AI-generated or real with reliable predictions.

Experimental Setup & Hyperparameters:

1. Hardware Setup:

The training process was executed primarily on Google Colab, which provided an accessible and GPU-enabled environment for deep learning experimentation. The GPU instances available in Colab, such as NVIDIA Tesla T4 and Tesla P100, were sufficient for handling the CNN model and the dataset used in this project. These GPUs provide high computational power and efficient parallel processing, enabling faster execution of convolution operations. The runtime included approximately 12–16 GB of RAM, which helped in managing batch processing and image preprocessing tasks without memory issues. Using GPU acceleration significantly reduced training time compared to CPU-only environments, allowing multiple experiments and fine-tuning within a limited time. This made the development process efficient and practical for iterative improvements.

2. Software Environment:

The software stack was built using a combination of machine learning, backend, and frontend tools that worked cohesively throughout the development cycle.

ML Environment:

- Python 3.x served as the core programming language.
- PyTorch and Torchvision were used for building and training the CNN model, handling datasets, and performing GPU-based computations.
- NumPy and PIL helped with image operations and tensor manipulation.
- Matplotlib and Scikit-learn supported evaluation plots and statistical analysis.

Backend Environment:

- Django was used to build the backend server, handle image uploads, and integrate the trained model for prediction.

Frontend Environment:

- HTML, CSS, and JavaScript were used to design a simple and responsive user interface for uploading images and displaying results.

The combination of these tools created a seamless pipeline from data preprocessing to real-time deployment.

3. Hyperparameter Tuning:

Hyperparameter tuning was an important part of optimizing the final model. Several experiments were performed to identify the best configurations:

- **Learning Rate Testing:** Values such as $1e-3$, $5e-4$, and $1e-5$ were tested. Higher rates caused unstable training, while lower ones led to slow convergence.
- **Batch Size Variations:** Batch sizes of 16, 32, and 64 were tested.
 - Small batch sizes increased noise in gradients
 - Larger batches reduced learning efficiency
- **Dropout Probability:** The dropout Values between 0.3 and 0.6 were tested to control overfitting. A value of 0.5 gave optimal results.
- **Epoch Count:** Experiments were conducted with 5, 10, and 15 epochs.
 - Fewer epochs resulted in underfitting
 - Too many epochs risked overfitting Ten epochs offered optimal results based on validation scores.
 - Ten epochs provided the best performance based on validation results.

This experimentation ensured that the model reached a stable accuracy without unnecessary training overhead.

4. Training Duration & Observations:

Training time varied depending on GPU availability:

- On GPU: Approximately 6–10 minutes per training run
- On CPU: Approximately 25–30 minutes per run

During model training, several important observations were made:

- Validation accuracy improved steadily across early epochs, indicating good feature learning.
- Training and validation losses followed similar downward trends, showing minimal overfitting.
- Performance stabilized around mid-epochs, with only minor improvements afterward.

These insights helped in selecting the best performing model checkpoint for deployment. The consistent behaviour across multiple runs confirmed that the selected architecture and hyperparameters were well optimized for the dataset.

Challenges Faced and Solutions:

1. Dataset Imbalance:

One of the major challenges faced during the development of this project was the imbalance between the two classes in the dataset. The number of AI-generated images was significantly higher than real images, which could cause the model to become biased toward predicting most inputs as AI-generated. This imbalance negatively affects the model's ability to correctly identify real images and reduces overall classification reliability.

Solution:

To address this issue, random undersampling was applied to the AI-generated class to balance it with the real image class. This ensured that both categories were equally represented during training, allowing the model to learn distinguishing features more effectively. In future work, more advanced techniques such as data augmentation, synthetic data generation, or collecting additional real-world images can be used to further improve dataset balance and model performance.

2. Subtle Difference Between AI and Real Images:

AI-generated images produced by modern tools are highly realistic and often difficult to distinguish from real photographs. They effectively replicate lighting conditions, textures, shadows, and fine visual details, making the classification task very challenging. The minimal visual differences between AI-generated and real images increase the complexity of the problem, even for deep learning models.

Solution:

To address this challenge, a Convolutional Neural Network (CNN) was trained to capture subtle visual patterns and hidden features present in images. Multiple training iterations were performed with proper hyperparameter tuning and dropout regularization to improve the model's ability to generalize. The model learns to identify fine-grained differences such as unnatural textures, irregular noise patterns, and structural inconsistencies that are more common in AI-generated images, thereby improving classification accuracy.

3. Overfitting During Training:

Due to the limited diversity and comparatively smaller number of real images, the model showed signs of overfitting during training. Overfitting occurs when the model performs very

well on training data but fails to generalize to unseen validation data. This indicated that the model was memorizing specific patterns rather than learning generalized features.

Solution:

To address overfitting, several techniques were applied:

- Introduced a Dropout layer with $p = 0.5$ to regularize the network
- Early Stopping was implemented to stop training when validation performance stopped improving
- Multiple training runs were conducted to identify stable learning behavior
- Learning rate was reduced to ensure smoother convergence

These techniques improved the model's ability to generalize and perform better on unseen data.

4. Hardware and Runtime Limitations:

Training deep learning models like CNN models requires significant computational resources. While using Google Colab, certain limitations were encountered such as limited session time, restricted GPU availability, and memory constraints. Larger batch sizes or prolonged training sessions sometimes led to runtime interruptions or memory overflow issues.

Solution:

To overcome these limitations, the training process was optimized using the following approaches:

- GPU runtimes were utilized whenever available
- Batch size was adjusted based on available memory
- GPU memory was cleared between training runs
- Model checkpoints were saved frequently to prevent data loss
- Training was distributed across multiple sessions when required

These optimizations ensured a smooth and efficient training process within the available computational constraints.

Future Scope:

The AI vs Real Image Detection System offers a strong foundation for media authenticity verification, but it has the potential to be expanded in many directions. The following subsections describe how the system can evolve to become more effective, scalable, and widely applicable.

1. Integration with Real-World Data Sources:

A major improvement for future work is connecting the system to real-world image sources through public APIs, such as social media platforms, news agency feeds, or online image repositories. By collecting real-time images from platforms like Twitter (X), Reddit, Instagram, or news networks, the model can continuously learn from new content trends, image formats, and evolving AI-generation techniques. This dynamic data stream would significantly improve the model's robustness and ensure that it adapts quickly to emerging AI tools and manipulation methods.

2. upgrade to Advanced Neural Architectures:

While the current CNN model performs effectively for binary classification, more advanced image-analysis architectures can be explored in the future. Models such as EfficientNet, Vision Transformers (ViT), or hybrid CNN-Transformer networks can extract deeper contextual and texture-based features, improving the system's ability to detect highly realistic AI-generated images. Implementing these architectures can enhance performance on complex and high-resolution image datasets.

3. Deployment as a Real-Time Web or Mobile Application:

To improve accessibility and real-world usability, the system can be deployed as:

- A full-scale web application with user accounts and history tracking
- A mobile application enabling instant image authenticity checks
- A browser extension that automatically scans images on websites or social media

Such deployments would allow users—including students, researchers, and general users—to quickly verify images in real-world scenarios, enhancing trust and reducing the spread of misinformation.

ML Pipeline Execution Stages :

1.Data Collection :

```
# Data source: Live webcam feed (real-time frames)

import cv2

cap = cv2.VideoCapture(0) # Capture from webcam|
```

2.Data Preprocessing :

```
import numpy as np
from PIL import Image
import io, base64

def frame_to_base64(frame_bgr, quality=85):
    rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
    pil = Image.fromarray(rgb)
    buf = io.BytesIO()
    pil.save(buf, format='JPEG', quality=quality)
    return base64.b64encode(buf.getvalue()).decode('utf-8')
```

3.Exploratory Data Analysis (EDA):

```
# Emotion distribution visualization via scores

EMOTION_EMOJI = {
    'happy': ('😊', 'Happy', (0, 210, 90)),
    'sad': ('😞', 'Sad', (255, 80, 60)),
    'angry': ('😡', 'Angry', (30, 40, 255)),
    'fear': ('😱', 'Fearful', (255, 200, 0)),
    'surprise': ('😮', 'Surprised', (0, 255, 255))
}
```

4.Feature Engineering / Selection :

```
# Extract facial emotion features using DeepFace

from deepface import DeepFace

def detect_emotion_from_frame(frame_bgr):
    result = DeepFace.analyze(
        frame_bgr,
        actions=['emotion'],
        enforce_detection=False,
        silent=True
    )

    if isinstance(result, list):
        result = result[0]

    emotion = result['dominant_emotion'].lower()
    scores = {k.lower(): v/100 for k, v in result['emotion'].items()}

    return emotion, scores
```

5.Model Training :

```
# No training required (pre-trained model)

# If DeepFace unavailable → fallback mock
import random

def mock_emotion():
    return random.choice(['happy', 'sad', 'angry', 'disgust', 'fear', 'surprise', 'neutral'])
```

6.Model Evaluation and Optimization :

```
# Use confidence scores from DeepFace

emotion, scores = detect_emotion_from_frame(frame)

print("Detected Emotion:", emotion)
print("Confidence Scores:", scores)
```

7. Deployment & MLOps :

```
# API integration for song recommendation

import requests

RAPIDAPI_KEY = "YOUR_API_KEY"

def search_tracks(query):
    url = "https://spotify23.p.rapidapi.com/search/"

    headers = {
        "X-RapidAPI-Key": RAPIDAPI_KEY,
        "X-RapidAPI-Host": "spotify23.p.rapidapi.com"
    }

    params = {"q": query, "type": "tracks", "limit": "5"}

    response = requests.get(url, headers=headers, params=params)
    data = response.json()

    return data
```

8. Monitoring & Maintenance :

```
# Real-time loop + smoothing for stable predictions

import collections

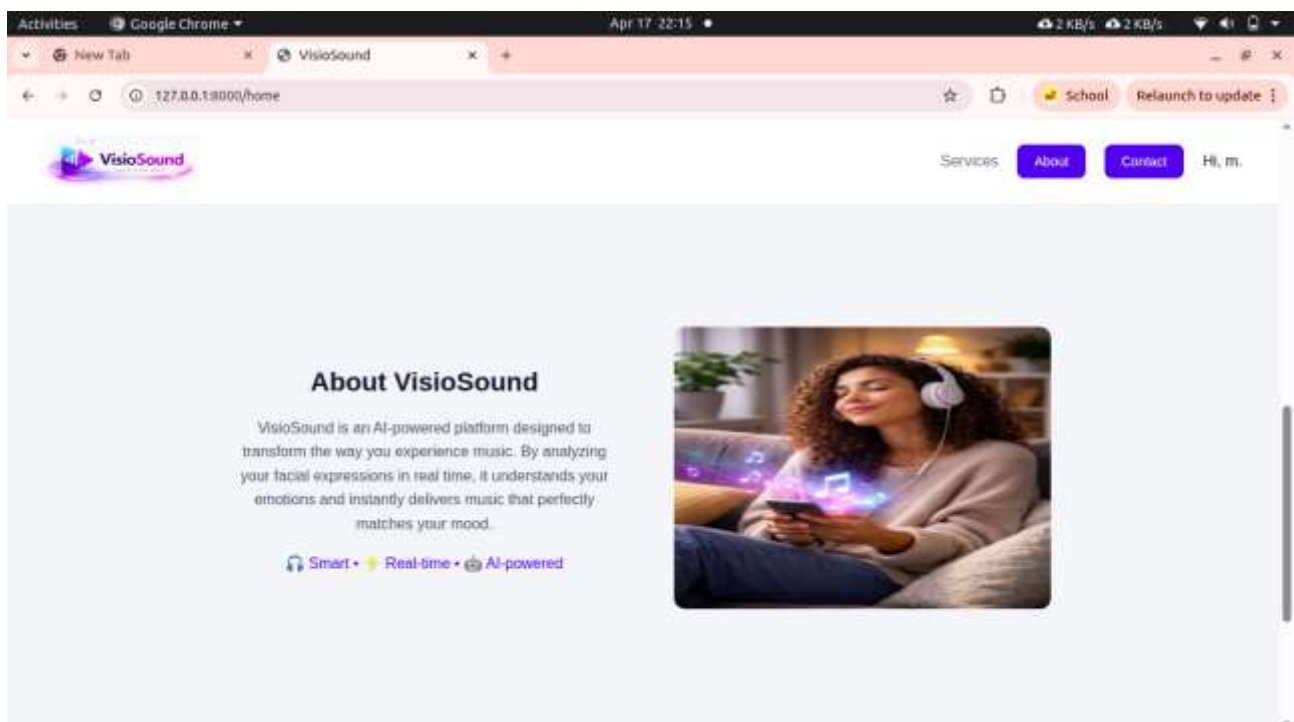
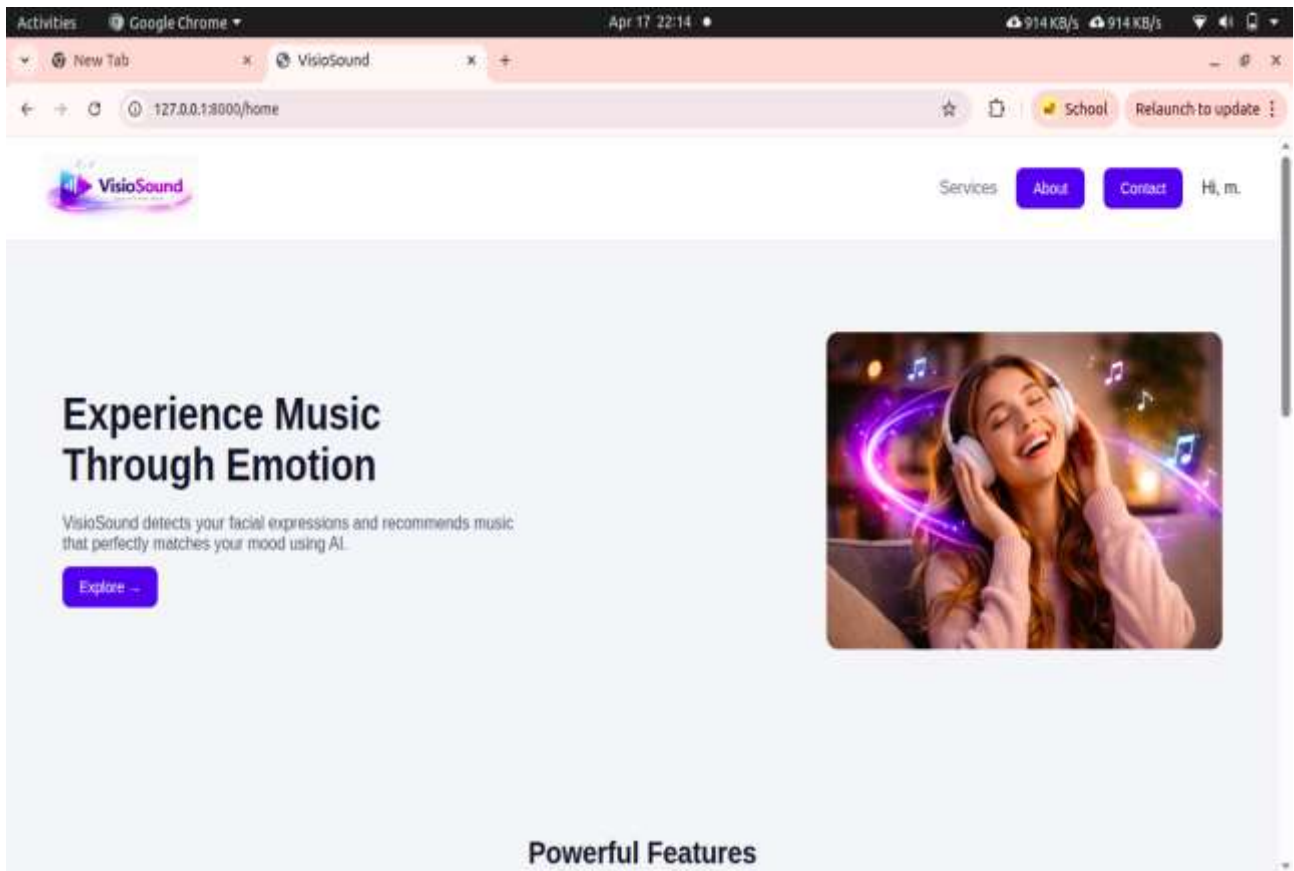
SMOOTH_WINDOW = 8
emotion_window = collections.deque(maxlen=SMOOTH_WINDOW)

emotion_window.append(emotion)

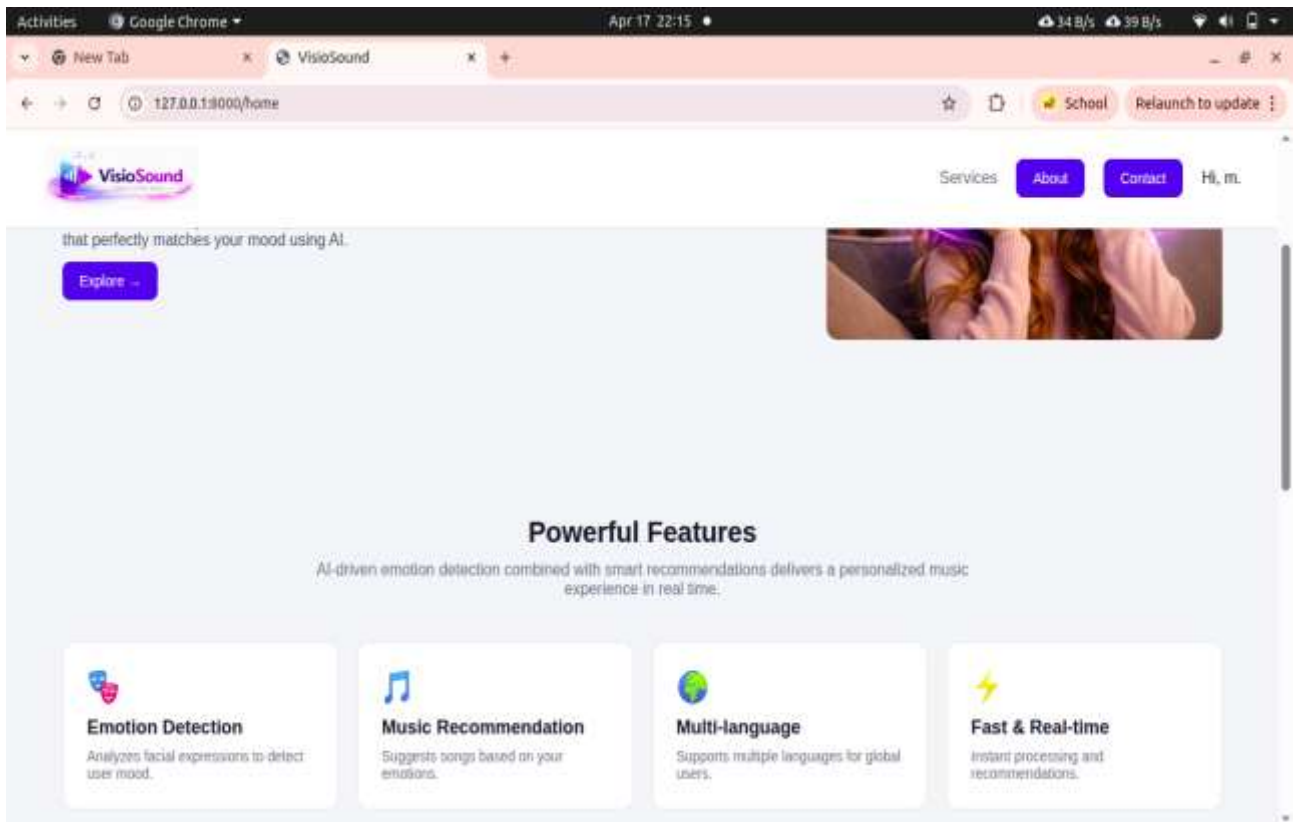
# Majority voting
final_emotion = max(set(emotion_window), key=emotion_window.count)
```

Visio Sound Website

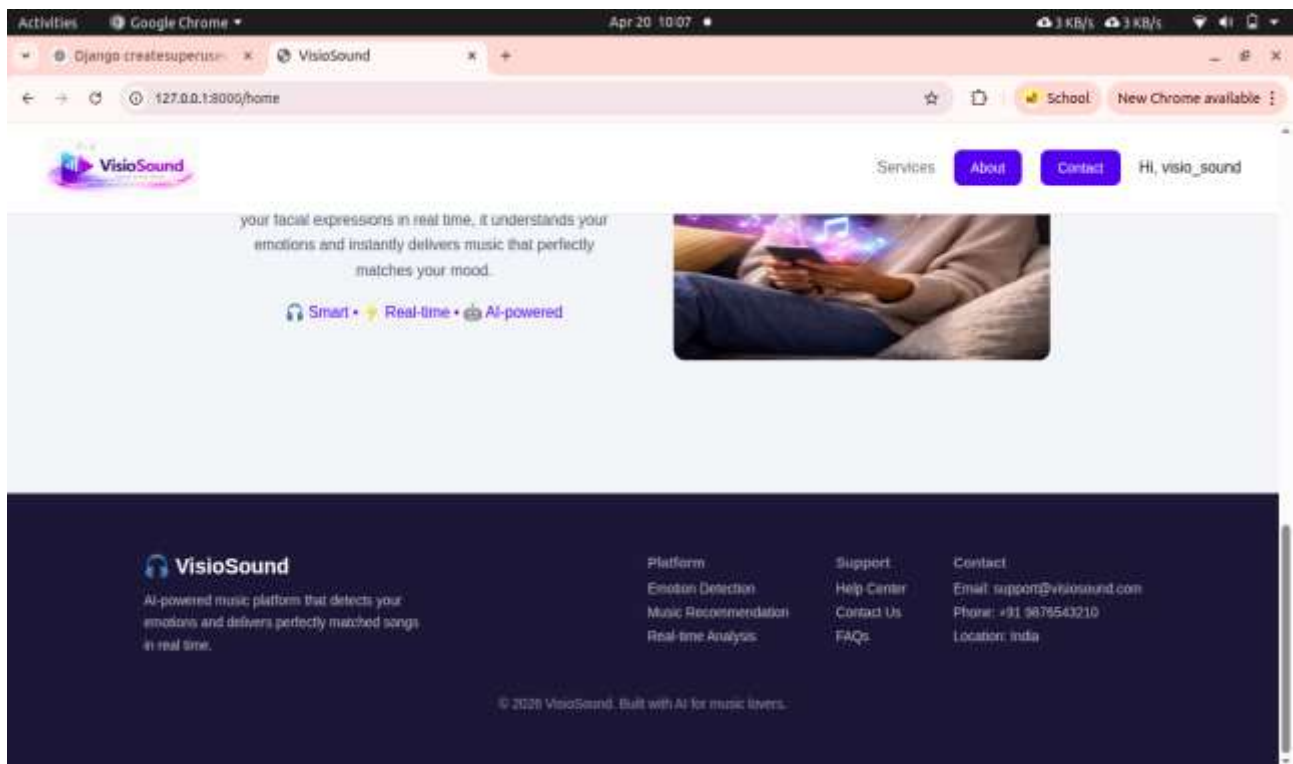
Landing page:



About a Website :



Footer and Contact Section:

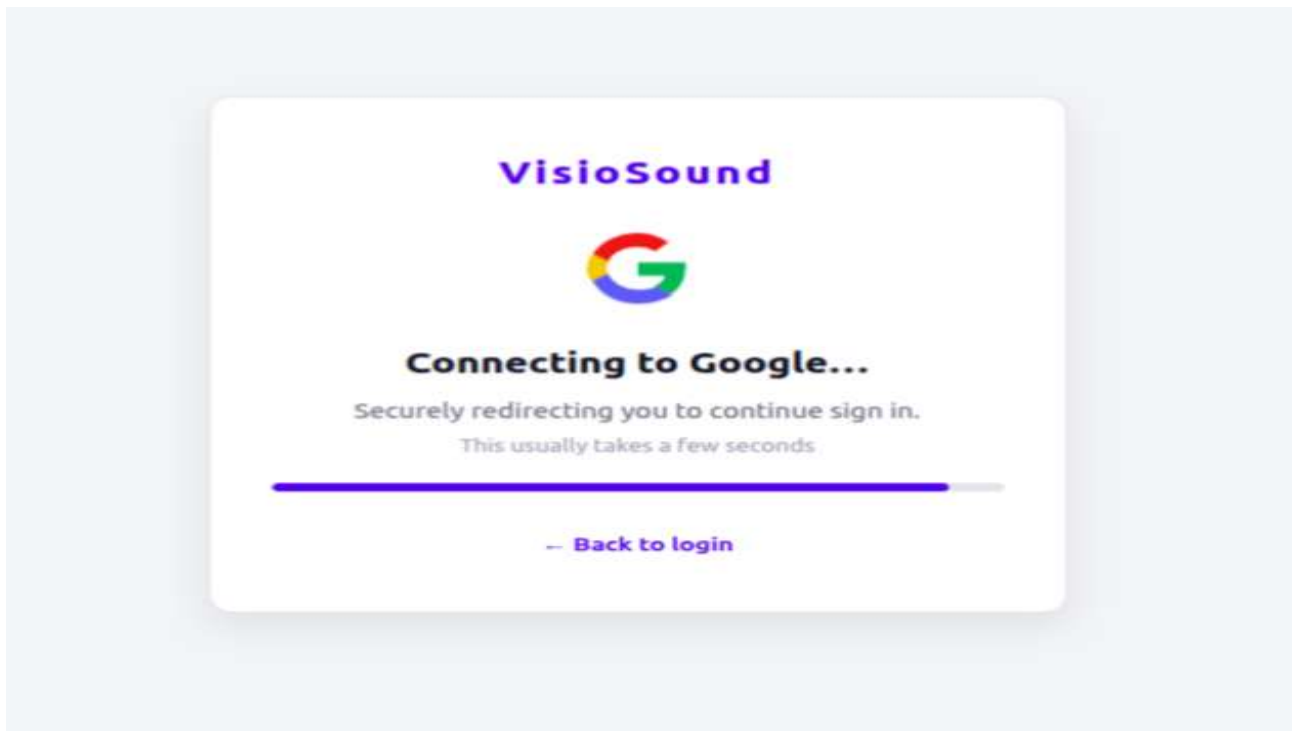


User Data Credentials :

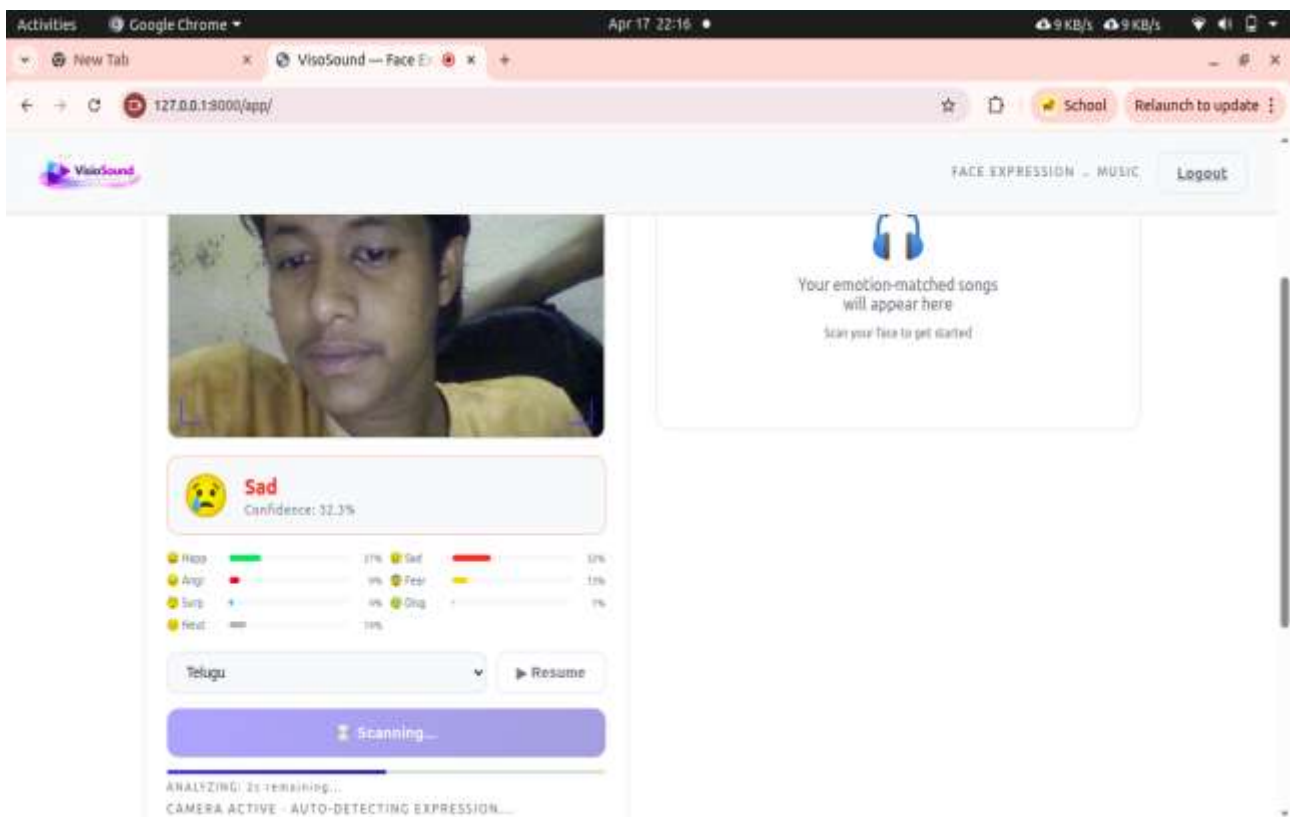
The image displays two side-by-side screenshots of the VisoSound user interface. The left screenshot shows the 'Create account' page with the tagline 'Your emotion. Your music.' It includes input fields for Username, Email, Preferred Music Language (set to English), Password, and Confirm Password. A purple 'Create Account' button is at the bottom, followed by a link to 'Continue with Google' and a 'Sign in' link for existing users. The right screenshot shows the 'Welcome Back' login page with the tagline 'Login to your account'. It features input fields for Username and Password, a purple 'Login' button, a 'Forgot password?' link, and a 'Continue with Google' button. A 'Create account' link is also present at the bottom.

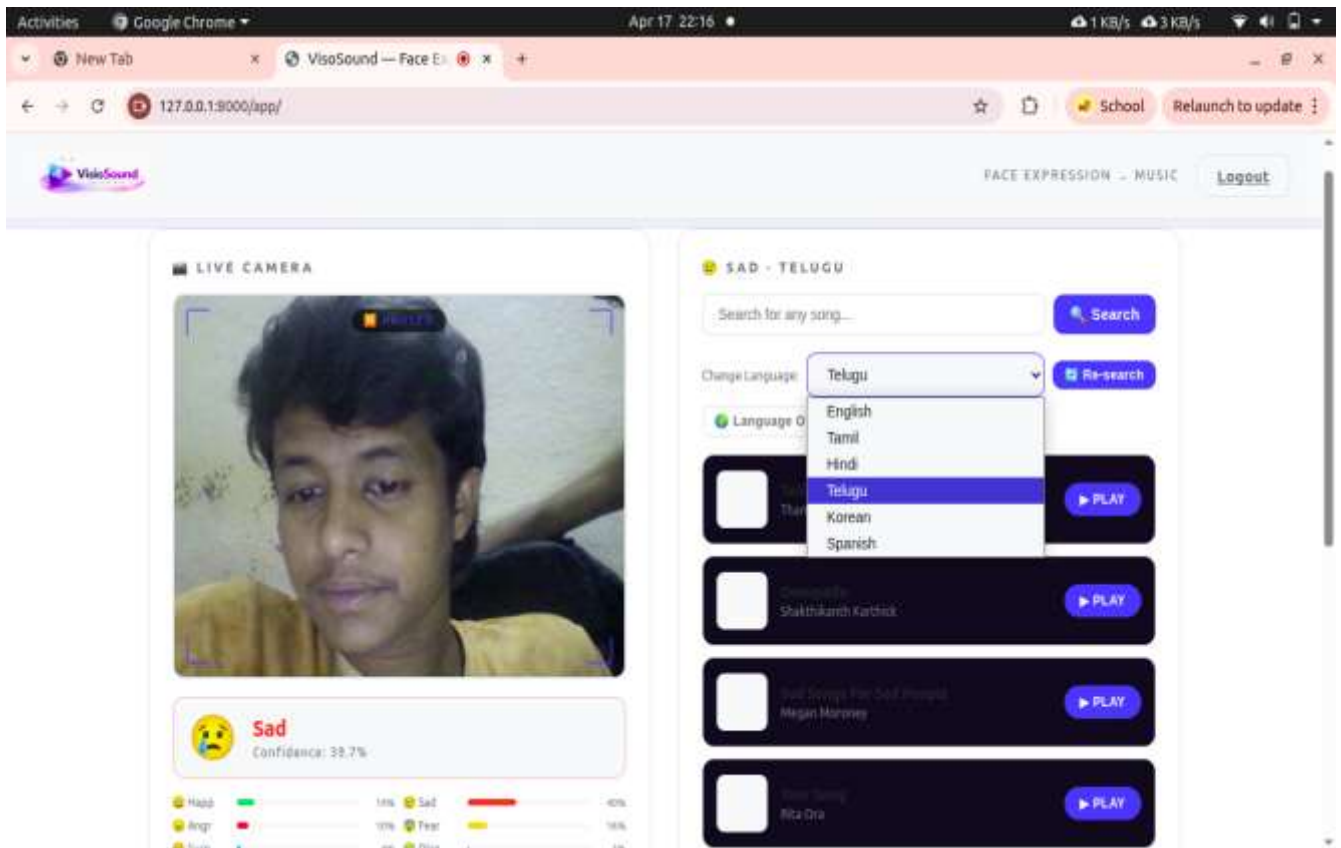
Google Oauth:

This screenshot shows the Google OAuth 'Choose an account' screen. At the top, it says 'Sign in with Google'. The main heading is 'Choose an account to continue to Viso_Sound'. On the right, two account options are listed: 'r210747@rguktrkv.ac.in' and 'M. Sathish' (msathish4386@gmail.com). Below these is a link to 'Use another account'. At the bottom left, the language is set to 'English (United States)', and at the bottom right, there are links for 'Help', 'Privacy', and 'Terms'.

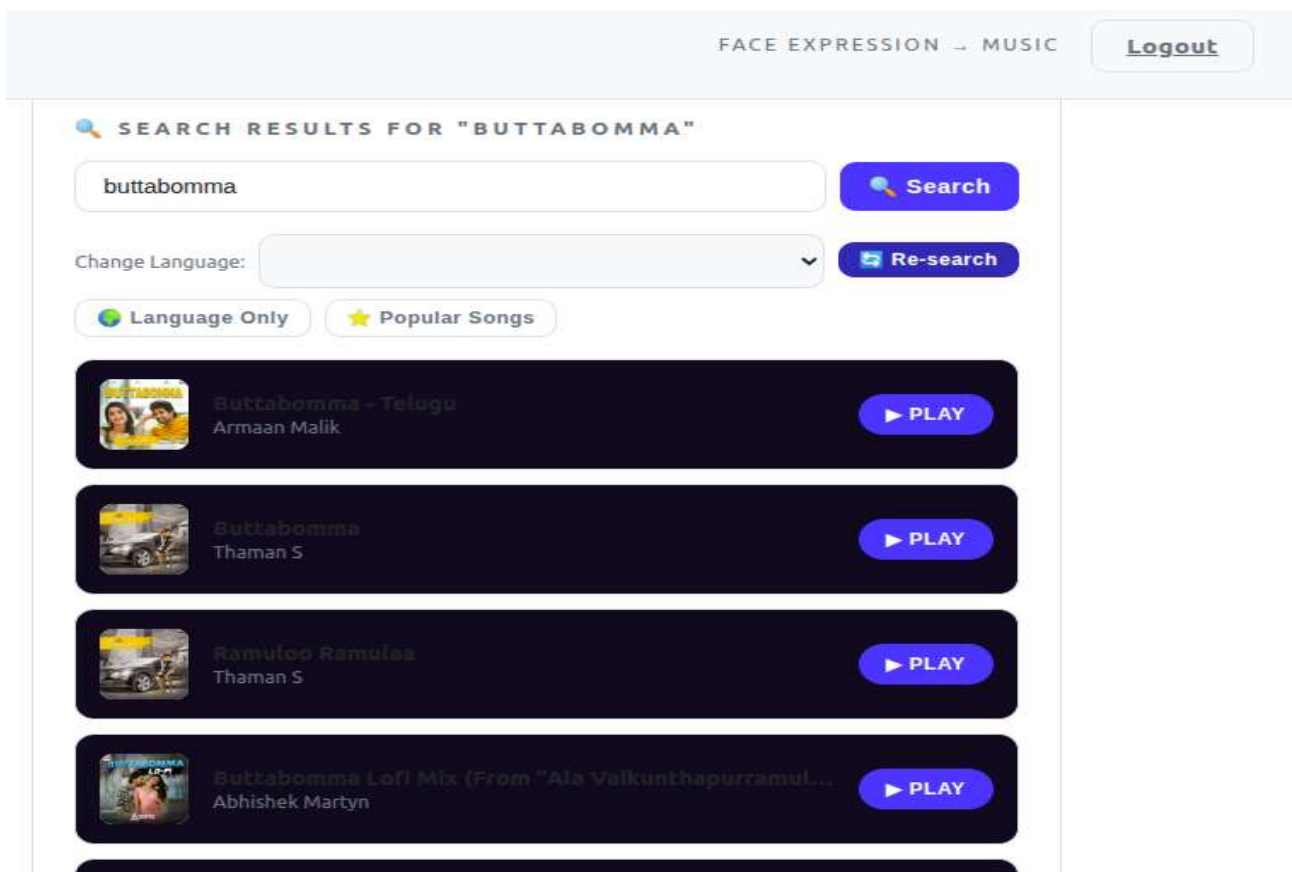


Music Recommendation System :-

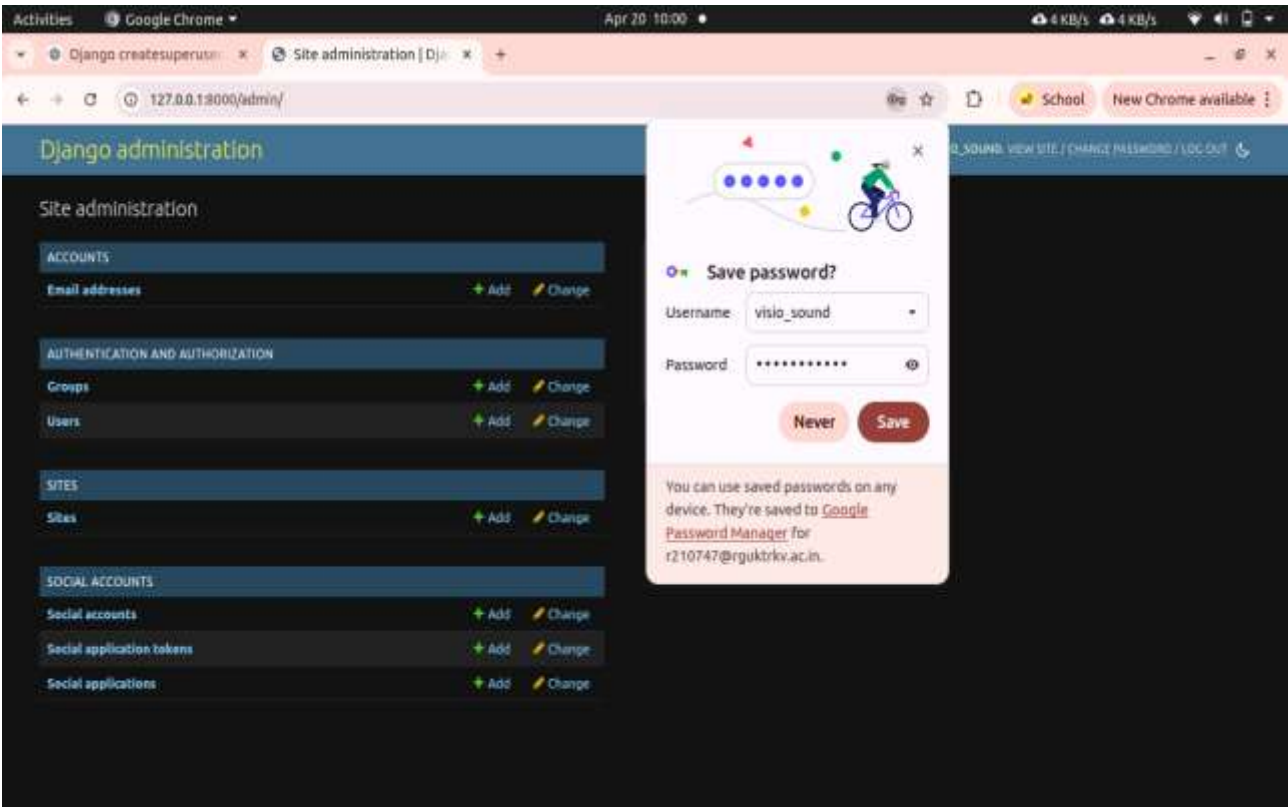
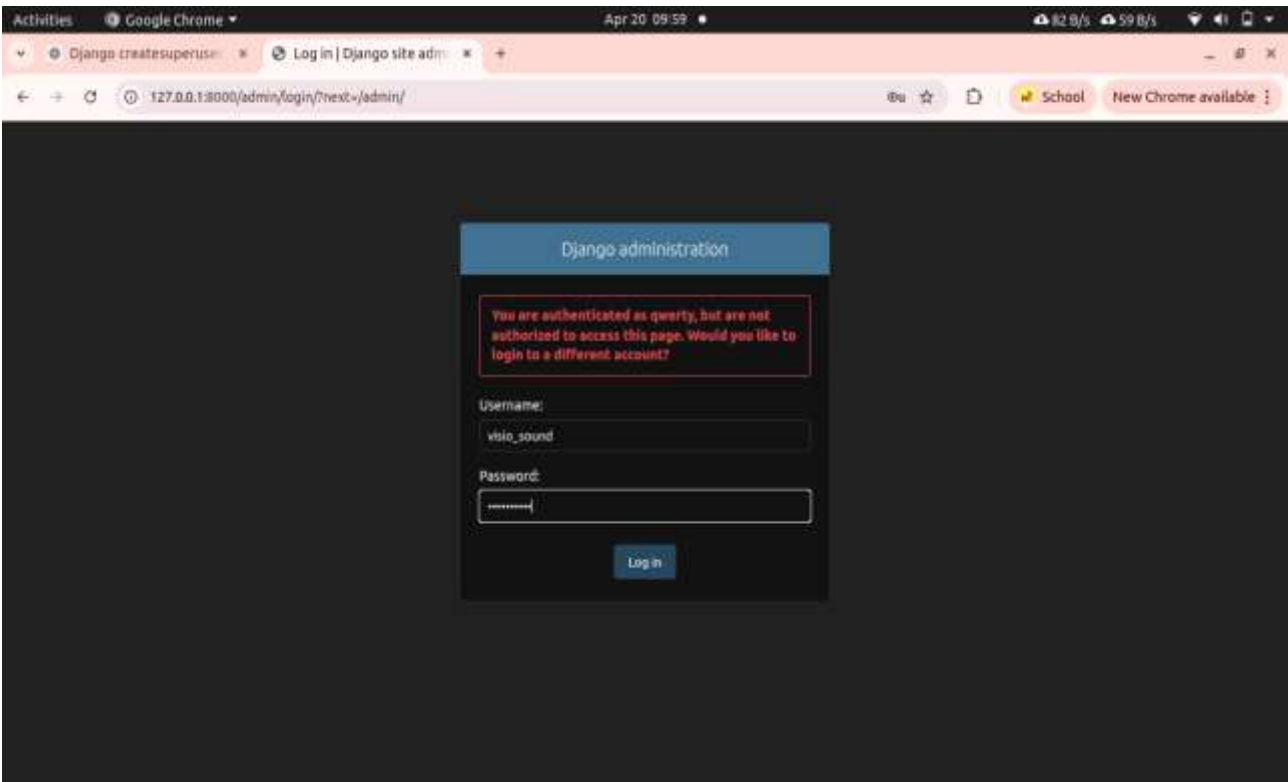


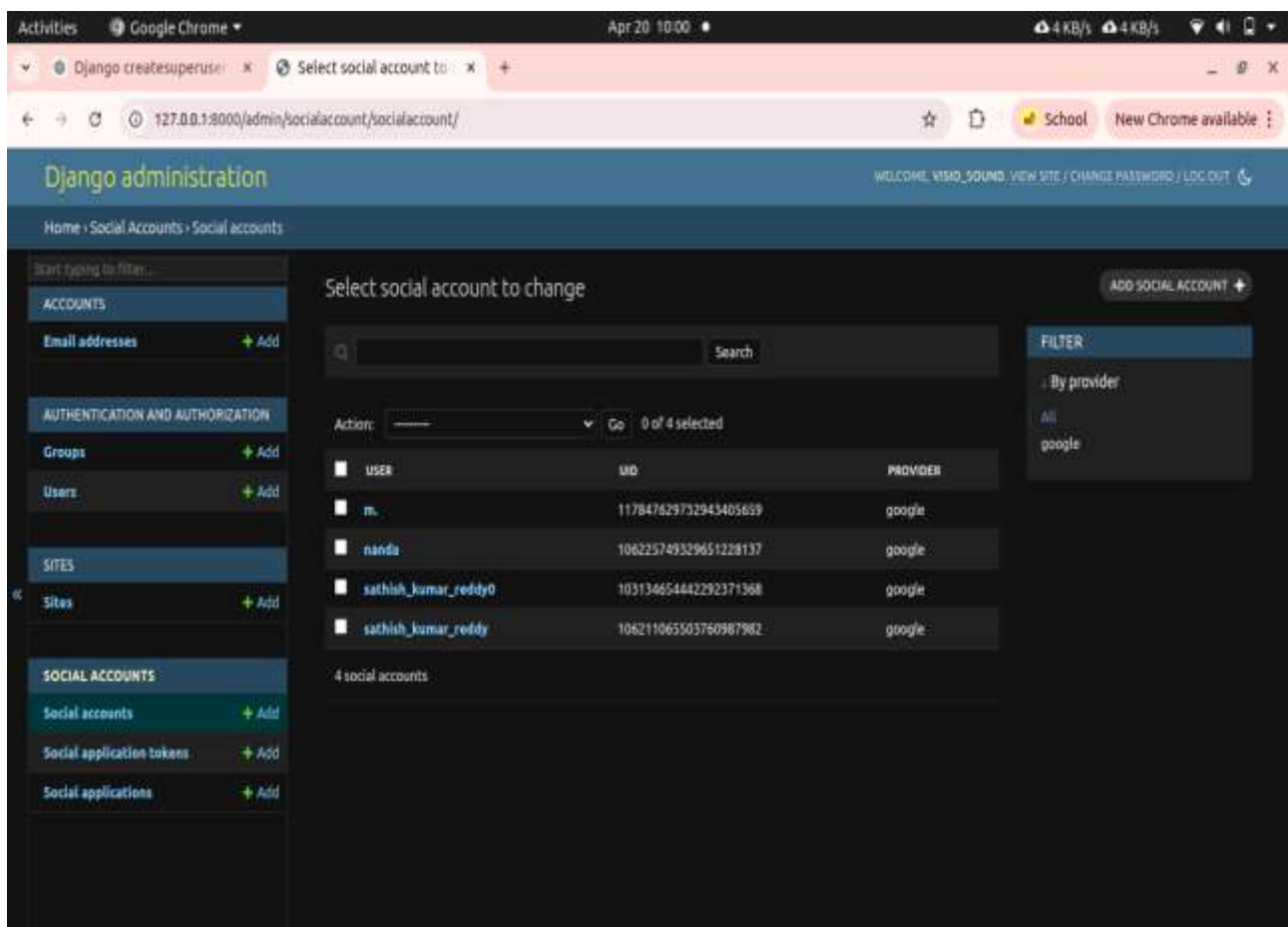
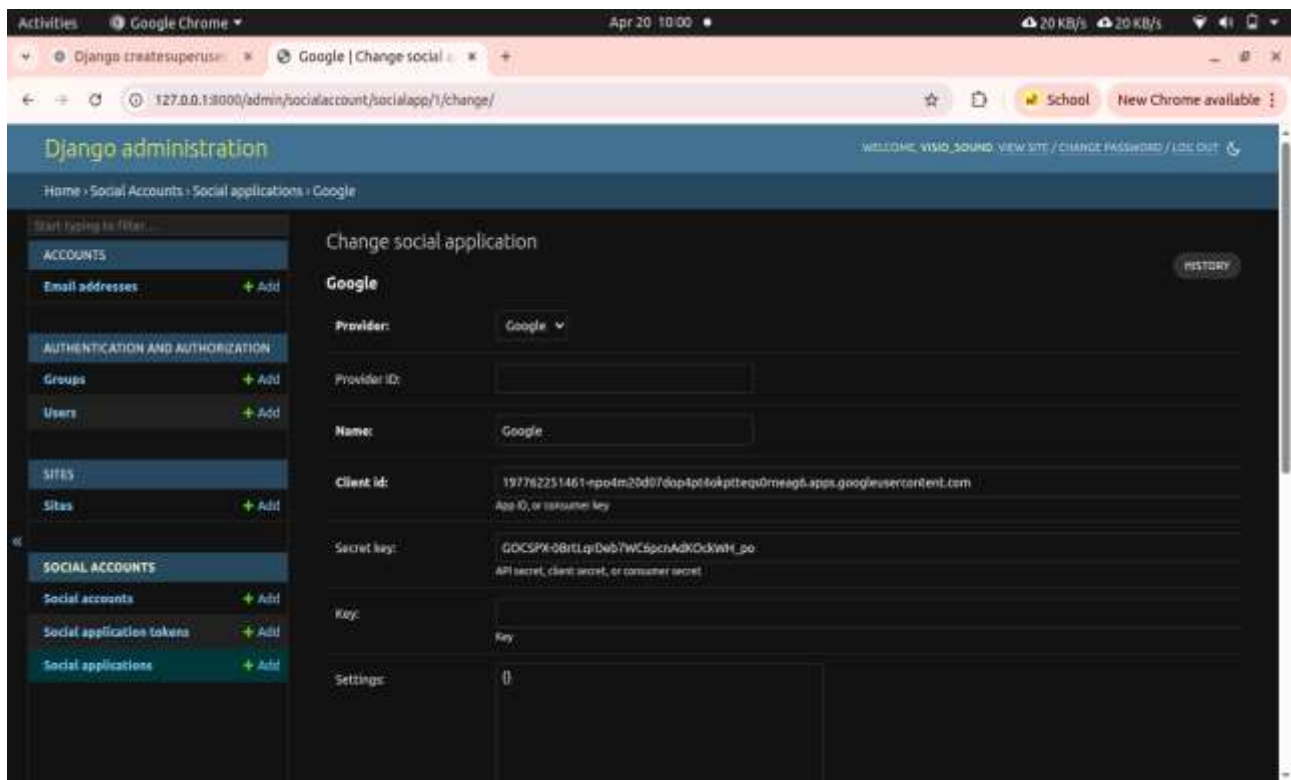


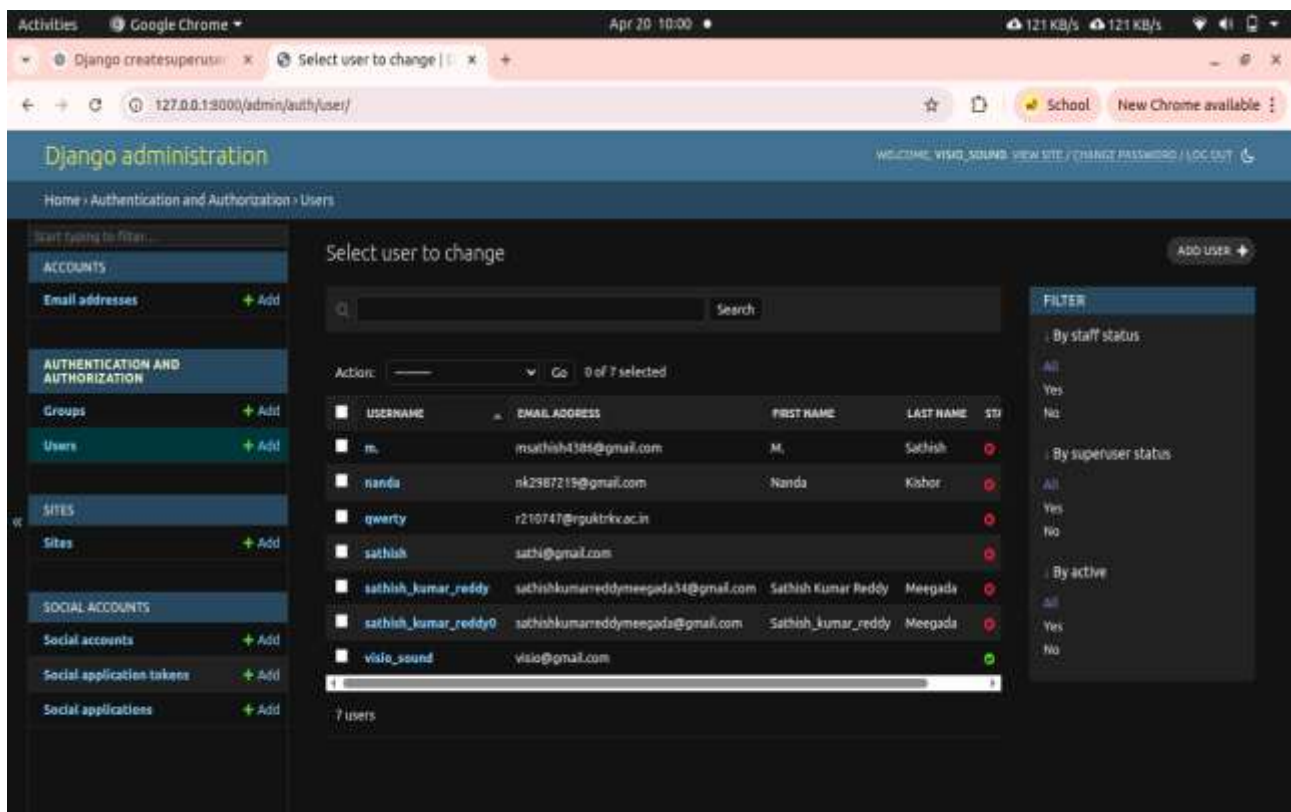
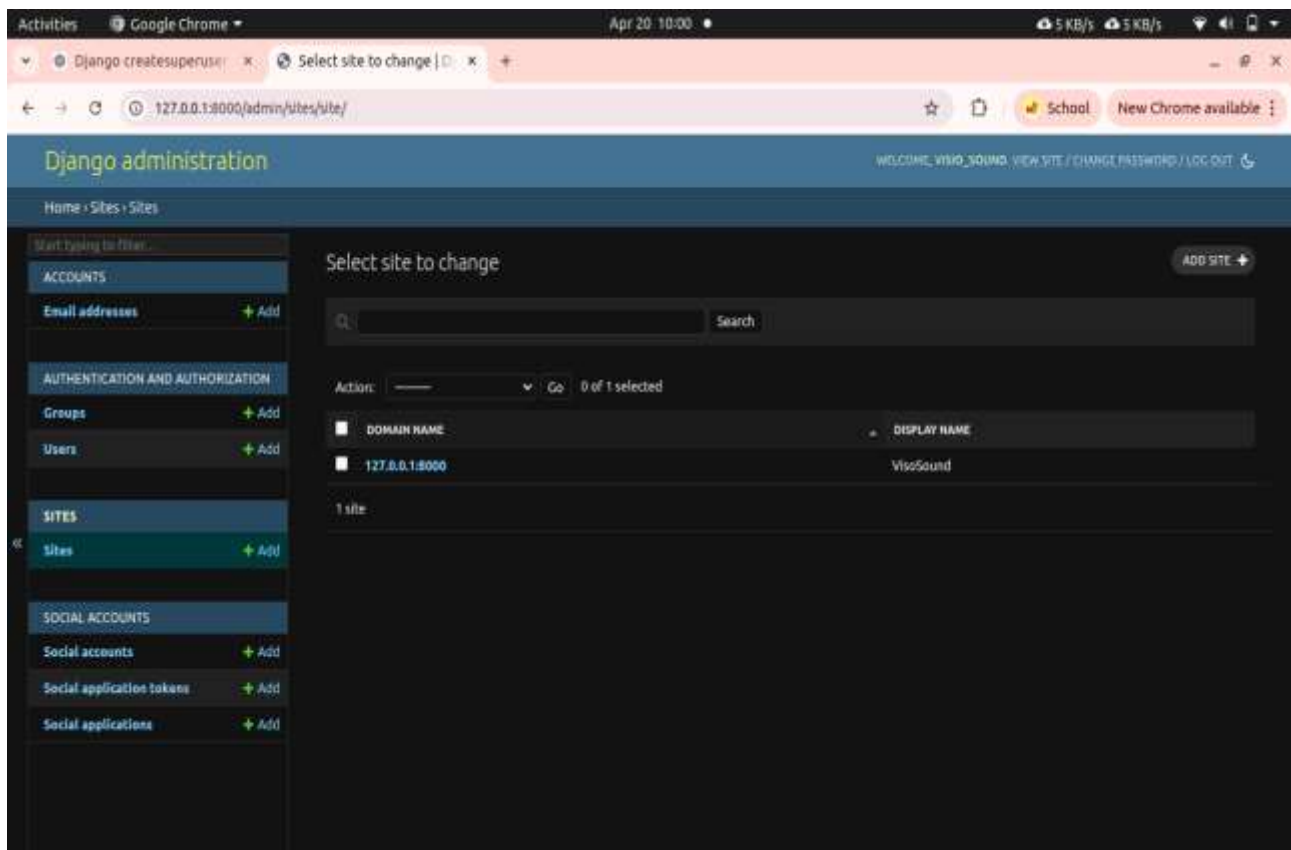
Browse songs :



Admin credentials :







Conclusion:

This project successfully presents the design and implementation of an Emotion-Based Music Recommendation System that leverages deep learning and computer vision techniques to enhance user experience. By utilizing a Convolutional Neural Network (CNN), the system is capable of accurately detecting human emotions from facial expressions in real time.

The developed system integrates multiple components, including image preprocessing, model training, emotion prediction, and a web-based interface, to deliver a seamless end-to-end solution. Through proper dataset preparation, normalization, and optimization techniques such as dropout and early stopping, the model achieved stable performance and reliable emotion classification.

Based on the detected emotion, the system recommends music tracks that align with the user's mood, thereby creating a personalized and engaging experience. The integration of the trained model with a Django-based backend and a responsive frontend demonstrates the practical applicability of artificial intelligence in real-world applications.

This project highlights the potential of emotion-aware systems in improving human-computer interaction. Although the current system performs effectively, there is scope for further enhancements such as improving model accuracy with larger datasets, integrating advanced deep learning architectures, and expanding recommendation capabilities using intelligent music APIs.

Overall, this work demonstrates how AI can be effectively used to bridge the gap between human emotions and digital entertainment, paving the way for more adaptive and personalized systems in the future.